TÀI LIỆU ÔN THI VÀ TRA CỨU ĐỀ MỞ: PHÁT TRIỂN HỆ THỐNG TÍCH HỢP

(Trọng tâm: Lập trình Socket TCP & UDP — RMI — JDBC — OOP Network)

⚡ PHẦN 0: MỤC LỤC TRA CỨU NHANH (5 GIÂY)

Phần	Nội dung trọng tâm	Vị trí tra cứu
I	**3 Bài toán Coding & OOP đặc biệt (Xác suất ra 100%)**	Phần I
II	**Thư viện 10 hàm CodeRunner chuẩn 100% (Viết tay dưới 15 dòng)**	Phần II
III	**Bảng tra cứu & Chi tiết 45 câu trắc nghiệm gốc (Xác suất ra 99%)**	Phần III
IV	**Ma trận Tradeoff so sánh cốt lõi (TCP vs UDP, Threading, I/O, JDBC)**	Phần IV
V	**Ngân hàng 10 sự cố mạng (Chuẩn 3 bước: Nguyên nhân ➔ Hậu quả ➔ Khắc phục)**	Phần V

🎯 PHẦN I: 3 BÀI TOÁN CODING & OOP ĐẶC BIỆT (XÁC SUẤT RA THI 100%)

1. Bài toán 1: Interface thay `void` bằng kiểu dữ liệu cụ thể (`String` và `int`) và Lớp triển khai

Yêu cầu & Bản chất: Khi phương thức trong Interface có kiểu trả về cụ thể (**String**, **int**...) thay vì **void**, lớp con triển khai (**implements**) **bắt buộc phải có lệnh `return`** trả về đúng kiểu dữ liệu đó, nếu không sẽ bị lỗi biên dịch *"missing return statement"*.

```
// 1. Định nghĩa Interface (không dùng void)
interface Animal {
    String makeSound();      // Trả về một chuỗi String thay vì void
    int getSleepHours();     // Trả về một số nguyên int thay vì void
}

// 2. Triển khai Interface trong một Class
class Dog implements Animal {
    @Override
    public String makeSound() {
        return "Woof! Woof!"; // Bắt buộc phải có lệnh return trả về chuỗi String
    }

    @Override
    public int getSleepHours() {
        return 8;             // Bắt buộc phải có lệnh return trả về số int
    }
}

// 3. Sử dụng trong hàm Main
public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        String sound = myDog.makeSound();
        int hours = myDog.getSleepHours();
        System.out.println("Sound: " + sound);      // In ra: Sound: Woof! Woof!
        System.out.println("Sleep: " + hours + "h"); // In ra: Sleep: 8h
    }
}
```

2. Bài toán 2: Lớp `ClientHandler` kế thừa `Thread` (`extends Thread`) dùng `BufferedStream`

Yêu cầu & Bản chất: Khi **ClientHandler extends Thread**, đối tượng chính là một luồng độc lập có sẵn phương thức **.start()**. Đọc mảng byte qua **BufferedInputStream**, bóc tách chuỗi UTF-8, nếu nhận lệnh **exit** thì dừng, đổi sang chữ IN HOA gửi lại qua **BufferedOutputStream** và gọi **.flush()** ngay lập tức.

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class ClientHandler extends Thread {
    private Socket socket;

    public ClientHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try {
            BufferedInputStream bis = new BufferedInputStream(socket.getInputStream());
```

```
            BufferedOutputStream bos = new BufferedOutputStream(socket.getOutputStream());
            byte[] buffer = new byte[1024];
            int bytesRead;

            while ((bytesRead = bis.read(buffer)) != -1) {
                String msg = new String(buffer, 0, bytesRead, StandardCharsets.UTF_8).trim();
                if (msg.equalsIgnoreCase("exit")) {
                    break;
                }
                String response = "Server trả lời: " + msg.toUpperCase() + "\n";
                bos.write(response.getBytes(StandardCharsets.UTF_8));
                bos.flush(); // Bắt buộc: Đẩy dữ liệu ra mạng ngay
            }
            socket.close();
        } catch (IOException e) {
            System.out.println("Client ngắt kết nối.");
        }
    }
}
```

3. Bài toán 3: Che khuất biến (`Variable Shadowing/Hiding`) vs Ghi đè phương thức (`Method Overriding`)

Đoạn mã phân tích:

```
class Vehicle {
    protected String brand = "Ford";
    public void show() { System.out.println("Method của Vehicle"); }
}
class Car extends Vehicle {
    protected String brand = "Toyota"; // Che khuất biến brand của Vehicle
    @Override
    public void show() { System.out.println("Method của Car"); }
}
public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car(); // Khai báo kiểu cha (Vehicle), khởi tạo đối tượng con (Car)
        v.show();              // In ra: "Method của Car"
        System.out.println(v.brand); // In ra: "Ford" (KHÔNG PHẢI Toyota!)
    }
}
```

Trả lời câu hỏi thi/vấn đáp: "Tại sao Java thiết kế như vậy?":

Với Phương thức (Method): Java áp dụng cơ chế **Dynamic Method Dispatch (Late Binding)**. JVM kiểm tra loại đối tượng thực tế trong vùng nhớ Heap tại thời điểm runtime (ở đây là **Car**), do đó phương thức **@Override** của lớp con được thực thi ➔ Đây chính là tính chất **Đa hình (Polymorphism)**.

Với Thuộc tính / Biến (Field / Variable): Java **KHÔNG áp dụng tính đa hình cho biến**, mà áp dụng cơ chế **Static Binding (Early Binding tại thời điểm compile-time)**. Trình biên dịch xác định ô nhớ và offset của biến dựa trên kiểu tham chiếu khai báo bên trái (**Vehicle**). Biến **brand** trong **Car** chỉ **che khuất (hiding)** chứ không ghi đè biến của **Vehicle**. Thiết kế này giúp tối ưu tốc độ truy xuất ô nhớ và bảo toàn tính đóng gói dữ liệu của lớp cha.

💻 PHẦN II: BỘ THƯ VIỆN 10 HÀM CODERUNNER CHUẨN 100% (VIẾT TAY DƯỚI 15 DÒNG)

1. Tạo DatagramPacket chứa text UTF-8 (makePacket)

```
java.net.DatagramPacket makePacket(String text, java.net.InetAddress destination, int port) {
    byte[] data = text.getBytes(java.nio.charset.StandardCharsets.UTF_8);
    return new java.net.DatagramPacket(data, data.length, destination, port);
}
```

2. Gửi một chuỗi UTF-8 qua UDP (sendText UDP)

```
void sendText(java.net.DatagramSocket socket, java.net.InetAddress address, int port, String message) throws
java.io.IOException {
    byte[] data = message.getBytes(java.nio.charset.StandardCharsets.UTF_8);
    java.net.DatagramPacket packet = new java.net.DatagramPacket(data, data.length, address, port);
    socket.send(packet);
}
```

3. Nhận một chuỗi UTF-8 từ UDP (receiveText UDP)

```
String receiveText(java.net.DatagramSocket socket) throws java.io.IOException {
    byte[] buffer = new byte[1024];
    java.net.DatagramPacket packet = new java.net.DatagramPacket(buffer, buffer.length);
    socket.receive(packet);
    return new String(packet.getData(), packet.getOffset(), packet.getLength(), java.nio.charset.StandardCharsets.UTF_8);
}
```

4. Gửi chuỗi và nhận chuỗi phản hồi qua UDP (requestUpper UDP)

```
String requestUpper(java.net.DatagramSocket socket, java.net.InetAddress address, int port, String message) throws
```

```
java.io.IOException {
    byte[] sendData = message.getBytes(java.nio.charset.StandardCharsets.UTF_8);
    socket.send(new java.net.DatagramPacket(sendData, sendData.length, address, port));
    byte[] receiveBuffer = new byte[1024];
    java.net.DatagramPacket receivePacket = new java.net.DatagramPacket(receiveBuffer, receiveBuffer.length);
    socket.receive(receivePacket);
    return new String(receivePacket.getData(), receivePacket.getOffset(), receivePacket.getLength(),
java.nio.charset.StandardCharsets.UTF_8);
}
```

5. Gửi một số double qua TCP (sendValue TCP)

```
void sendValue(java.net.Socket socket, double value) throws java.io.IOException {
    java.io.DataOutputStream dos = new java.io.DataOutputStream(socket.getOutputStream());
    dos.writeDouble(value);
    dos.flush(); // Bắt buộc flush, không đóng socket
}
```

6. Gửi một chuỗi bằng writeUTF qua TCP (sendText TCP)

```
void sendText(java.net.Socket socket, String message) throws java.io.IOException {
    java.io.DataOutputStream dos = new java.io.DataOutputStream(socket.getOutputStream());
    dos.writeUTF(message);
    dos.flush();
}
```

7. Đọc một chuỗi bằng readUTF từ TCP (receiveText TCP)

```
String receiveText(java.net.Socket socket) throws java.io.IOException {
    java.io.DataInputStream dis = new java.io.DataInputStream(socket.getInputStream());
    return dis.readUTF();
}
```

8. Gửi int và nhận int phản hồi qua TCP (requestDouble TCP)

```
int requestDouble(java.net.Socket socket, int value) throws java.io.IOException {
    java.io.DataOutputStream dos = new java.io.DataOutputStream(socket.getOutputStream());
    dos.writeInt(value);
    dos.flush();
    java.io.DataInputStream dis = new java.io.DataInputStream(socket.getInputStream());
    return dis.readInt();
}
```

9. Nhận lệnh in hoa và dừng khi gặp BYE qua TCP (processCommands TCP)

```
void processCommands(java.net.Socket socket) throws java.io.IOException {
    java.io.DataInputStream dis = new java.io.DataInputStream(socket.getInputStream());
    java.io.DataOutputStream dos = new java.io.DataOutputStream(socket.getOutputStream());
    while (true) {
        String cmd = dis.readUTF();
        if ("BYE".equals(cmd)) {
            dos.writeUTF("BYE");
            dos.flush();
            break;
        } else {
            dos.writeUTF(cmd.toUpperCase(java.util.Locale.ROOT));
            dos.flush();
        }
    }
}
```

10. Đọc số nguyên, tính bình phương hoặc báo INVALID (handleOne TCP)

```
void handleOne(java.net.Socket socket) throws java.io.IOException {
    java.io.DataInputStream dis = new java.io.DataInputStream(socket.getInputStream());
    java.io.DataOutputStream dos = new java.io.DataOutputStream(socket.getOutputStream());
    int val = dis.readInt();
    if (val >= 0 && val <= 9) {
        dos.writeUTF("SQUARE=" + (val * val));
    } else {
        dos.writeUTF("INVALID");
    }
    dos.flush();
}
```

11. Hai khung mã điền từ (Dropdown / Fill-in)

Mẫu 1 (Server đón Client - oneClient):

```
static void oneClient() throws IOException {
    try (ServerSocket server = new ServerSocket(5000)) {     // [Ô 1: ServerSocket]
```

```
        try (Socket client = server.accept()) {             // [Ô 2: Socket] | [Ô 3: accept]
            System.out.println(client.getRemoteSocketAddress());
        }
    }
}
```

Mẫu 2 (Client gửi nhận int qua DataStream - exchangeInt):

```
static int exchangeInt(Socket socket, int value) throws IOException {
    DataOutputStream out = new DataOutputStream(socket.getOutputStream());
    DataInputStream in = new DataInputStream(socket.getInputStream());
    out.writeInt(value);  // [Ô 1: writeInt]
    out.flush();          // [Ô 2: flush]
    return in.readInt();  // [Ô 3: readInt]
}
```

🏆 PHẦN III: BẢNG TRA CỨU & CHI TIẾT 45 CÂU TRẮC NGHIỆM GỐC (XÁC SUẤT RA LẠI 99%)

1. Bảng tra cứu tức thì 45 câu (Tra cứu 3 - 5 giây)

STT	Từ khóa nhận diện đề bài	Đáp án	Nội dung cốt lõi cần chọn
1	Đọc từng dòng UTF-8 từ socket	**B**	**InputStreamReader và BufferedReader**
2	Loại dữ liệu TCP cung cấp	**D**	**Dòng byte tin cậy và có thứ tự**
3	Hạn chế của server tuần tự	**A**	**Một client chậm có thể làm các client khác phải chờ**
4	Lý do phải đóng Socket	**B**	**Để giải phóng tài nguyên và kết thúc phiên**
5	Ưu tiên của luồng chính (Main Thread)	**C**	**Nhận kết nối và giao Socket cho worker**
6	Kiểm thử tính độc lập giữa các phiên	**D**	**Đóng đột ngột một client trong khi các client khác tiếp tục nhận phản hồi**
7	Đối tượng accept() trả về	**B**	**Một Socket của phiên kết nối**
8	Ý nghĩa khi readLine() trả về null	**B**	**Đầu bên kia đã đóng luồng**
9	Mục đích dùng println() gửi request	**B**	**Để tạo ranh giới thông điệp bằng ký tự xuống dòng**
10	Thời điểm server quay lại accept()	**A**	**Sau khi xử lý xong phiên client hiện tại**
11	Thành phần mở port & lắng nghe	**C**	**ServerSocket**
12	Lợi ích của Fixed Thread Pool	**A**	**Giới hạn số thread hoạt động và tái sử dụng thread**
13	Tác dụng của tham số true trong PrintWriter	**C**	**Bật autoFlush cho println(), printf() và format()**
14	Cách xử lý lỗi IOException của một client	**D**	**Bắt lỗi trong worker và tiếp tục phục vụ client khác**
15	Tổ chức request-response đúng	**D**	**Client gửi một dòng và chờ một dòng phản hồi**
16	UDP Server lấy IP và Port phản hồi từ đâu	**D**	**requestPacket.getAddress() và requestPacket.getPort()**
17	Chống trừ tiền 2 lần (Idempotency)	**D**	**Gắn mã request duy nhất, lưu kết quả đã xử lý và trả lại kết quả cũ khi request bị lặp**
18	Quá thời gian setSoTimeout(2000)	**B**	**receive() ném SocketTimeoutException nhưng socket vẫn có thể tiếp tục sử dụng**
19	Datagram lớn hơn buffer nhận	**B**	**Datagram bị cắt theo kích thước vùng nhận và phần vượt quá không được giữ lại**
20	Gói UDP lớn bị mất thông điệp	**B**	**Chia dữ liệu thành các datagram nhỏ hơn, gắn mã thông điệp và số thứ tự để kiểm tra và ghép lại**
21	Lớp gửi và nhận datagram UDP	**C**	**DatagramSocket**
22	Tái sử dụng DatagramPacket sau khi nhận gói nhỏ	**C**	**Gọi packet.setLength(buffer.length)**
23	Giải mã UTF-8 từ DatagramPacket	**A**	**new String(packet.getData(), packet.getOffset(), packet.getLength(), StandardCharsets.UTF_8)**
24	Gọi receive() khi chưa có datagram đến	**B**	**Phương thức tiếp tục chờ cho đến khi**

			nhận được datagram
25	Mô tả đúng nhất về UDP	**B**	**UDP là giao thức không kết nối và truyền dữ liệu theo datagram**
26	Lệnh tạo UDP server tại port 9876	**C**	**new DatagramSocket(9876)**
27	Giá trị cho biết số byte thực nhận UDP	**B**	**packet.getLength()**
28	Bảo đảm mà UDP KHÔNG cung cấp	**C**	**Dữ liệu được nhận đầy đủ và đúng thứ tự**
29	Đối tượng chứa dữ liệu và địa chỉ gói UDP	**C**	**DatagramPacket**
30	Cú pháp tạo gói DatagramPacket gửi đi	**D**	**new DatagramPacket(data, data.length, address, port)**
31	Câu 4: Client thực hiện hai lệnh:	**C**	**Server nhận một dòng byte; ranh giới giữa HELLO và WORLD không được TCP bảo đảm.**
32	Câu 5: Trong chương trình TCP, phương thức sau trả về thành phần nào?	**D**	**Luồng đọc dữ liệu do đầu bên kia gửi đến.**
33	Câu 6: Xét đoạn mã:	**A**	**Luồng đầu vào đạt EOF do đầu bên kia đóng luồng hoặc kết nối.**
34	Câu 7: Xét đoạn mã:	**D**	**Luồng chính nhận kết nối và giao từng Socket cho worker xử lý.**
35	Câu 8: Xét đoạn mã:	**D**	**Tự động đẩy dữ liệu khi gọi println().**
36	Điểm cuối gửi nhận UDP	**C**	**DatagramSocket**
37	Câu 10: Xét dòng lệnh:	**C**	**Chờ client kết nối và trả về Socket của phiên kết nối.**
38	Câu 11: Xét đoạn mã:	**A**	**receive() ném SocketTimeoutException và socket vẫn có thể sử dụng.**
39	Câu 12: Vì sao nên giải mã datagram bằng đoạn mã sau?	**A**	**Để chỉ giải mã vùng byte thực nhận bằng đúng bảng mã UTF-8.**
40	Câu 13: UDP client gửi yêu cầu trừ tiền và gửi lại request nếu hết thời gian chờ. Giải pháp nào giúp server không thực hiện cùng một giao dịch hai lần?	**A**	**Gắn mã request duy nhất và lưu kết quả của request đã xử lý.**
41	Câu 14: Xét đoạn mã phía UDP server:	**C**	**Xác định địa chỉ IP và port của client đã gửi request.**
42	Câu 15: Lớp Java nào được sử dụng để mở port và lắng nghe kết nối TCP ở phía server?	**D**	**ServerSocket**
43	Xây dựng UDP tin cậy (Reliable UDP)	**A**	**Bổ sung số thứ tự hoặc mã request, ACK, timeout và cơ chế retransmission.**
44	Câu 17: Một DatagramPacket có buffer 1024 byte vừa nhận datagram dài 100 byte. Trước khi tái sử dụng packet để nhận datagram tiếp theo, nên thực hiện lệnh nào?	**D**	**packet.setLength(buffer.length);**
45	Phân biệt cốt lõi giữa TCP và UDP	**C**	**TCP cung cấp dòng byte tin cậy; UDP truyền datagram không kết nối.**

2. Chi tiết toàn bộ 45 câu hỏi kèm lời giải thích bản chất

Câu 1: Câu 1: Cặp lớp nào phù hợp để đọc từng dòng UTF-8 từ socket?

A. DataOutputStream và PrintWriter

B. InputStreamReader và BufferedReader

C. FileReader và Scanner

D. ServerSocket và BufferedWriter

➔ Đáp án đúng: B. InputStreamReader và BufferedReader

Giải thích cốt lõi: InputStreamReader giúp chuyển đổi dòng byte từ socket (InputStream) thành các ký tự theo bảng mã UTF-8. BufferedReader cung cấp phương thức readLine() giúp đọc dữ liệu theo từng dòng hiệu quả.

Câu 2: Câu 2: TCP cung cấp loại dữ liệu nào cho ứng dụng?

A. Datagram không kết nối

B. Các thông điệp giữ nguyên ranh giới

C. Các đối tượng Java tự tuần tự hóa

D. Dòng byte tin cậy và có thứ tự

➔ **Đáp án đúng: D. Dòng byte tin cậy và có thứ tự**

Giải thích cốt lõi: TCP là giao thức hướng kết nối (connection-oriented), đảm bảo dữ liệu truyền đi theo dòng byte (byte stream) đáng tin cậy, không bị mất mát và đúng thứ tự.

Câu 3: Câu 3: Hạn chế chính của server tuần tự là gì?

A. Một client chậm có thể làm các client khác phải chờ

B. Không thể dùng UTF-8

C. Không thể đóng Socket

D. Không thể mở port

➔ **Đáp án đúng: A. Một client chậm có thể làm các client khác phải chờ**

Giải thích cốt lõi: Server tuần tự xử lý từng yêu cầu một. Nếu một client xử lý chậm hoặc bị treo, toàn bộ các client kết nối sau phải chờ cho đến khi client trước hoàn thành.

Câu 4: Câu 4: Tại sao phải đóng Socket sau khi hoàn thành?

A. Để tạo thêm request

B. Để giải phóng tài nguyên và kết thúc phiên

C. Để giữ port của client mãi mãi

D. Để thay đổi địa chỉ IP

➔ **Đáp án đúng: B. Để giải phóng tài nguyên và kết thúc phiên**

Giải thích cốt lõi: Việc đóng Socket (chứa các cổng kết nối và các luồng I/O) giúp hệ điều hành và JVM giải phóng bộ nhớ, file descriptors và tài nguyên mạng.

Câu 5: Câu 5: Trong server nhiều client, luồng chính nên ưu tiên công việc nào?

A. Đọc bàn phím của client

B. Xử lý toàn bộ request của một client

C. Nhận kết nối và giao Socket cho worker

D. Tạo lại JVM

➔ **Đáp án đúng: C. Nhận kết nối và giao Socket cho worker**

Giải thích cốt lõi: Trong mô hình Server đa luồng, luồng chính (Main thread) lắng nghe kết nối mới qua accept(), sau đó ngay lập tức chuyển Socket đó cho luồng worker xử lý để tránh làm gián đoạn việc tiếp nhận client mới.

Câu 6: Câu 6: Ca kiểm thử nào chứng minh tốt nhất tính độc lập giữa các phiên?

A. Đổi màu chữ trong IDE

B. Biên dịch server hai lần

C. Chạy một client gửi PING

D. Đóng đột ngột một client trong khi các client khác tiếp tục nhận phản hồi

➔ **Đáp án đúng: D. Đóng đột ngột một client trong khi các client khác tiếp tục nhận phản hồi**

Giải thích cốt lõi: Tính độc lập thể hiện ở việc sự cố hoặc kết thúc phiên của một client này không ảnh hưởng đến trạng thái cũng như việc truyền nhận dữ liệu của các client khác.

Câu 7: Câu 7: Phương thức accept() trả về đối tượng nào khi có client kết nối?

A. Một InputStream

B. Một Socket của phiên kết nối

C. Một Thread

D. Một ServerSocket mới

➔ **Đáp án đúng: B. Một Socket của phiên kết nối**

Giải thích cốt lõi: Khi một client kết nối thành công, ServerSocket.accept() sẽ trả về một đối tượng Socket đại diện cho kết nối riêng biệt giữa Server và Client đó.

Câu 8: Câu 8: Khi readLine() trả về null, tình huống nào phù hợp nhất?

A. Dòng nhận được rỗng

B. Đầu bên kia đã đóng luồng

C. Port đang bận

D. Không có dữ liệu trong một mili giây

➔ **Đáp án đúng: B. Đầu bên kia đã đóng luồng**

Giải thích cốt lõi: Trong BufferedReader, phương thức readLine() trả về null khi đạt đến cuối luồng (EOF - End Of Stream), tức là phía gửi đã đóng luồng hoặc đóng Socket.

Câu 9: Câu 9: Vì sao ví dụ dùng println() để gửi mỗi request?

A. Để mã hóa dữ liệu thành số

B. Để tạo ranh giới thông điệp bằng ký tự xuống dòng

C. Để tạo thêm một socket

D. Để thay thế accept()

➔ **Đáp án đúng: B. Để tạo ranh giới thông điệp bằng ký tự xuống dòng**

Giải thích cốt lõi: println() tự động thêm ký tự xuống dòng (\n hoặc \r\n) vào cuối chuỗi, giúp bên nhận dùng readLine() nhận biết chính xác điểm kết thúc của một gói tin/thông điệp.

Câu 10: Câu 10: Trong server tuần tự, khi nào server có thể quay lại accept()?

A. Sau khi xử lý xong phiên client hiện tại

B. Chỉ sau khi khởi động lại JVM

C. Ngay sau khi client kết nối

D. Trước khi tạo ServerSocket

➔ Đáp án đúng: A. Sau khi xử lý xong phiên client hiện tại

Giải thích cốt lõi: Server tuần tự chỉ dùng 1 luồng đơn duy nhất. Nó chỉ có thể quay lại vòng lặp gọi accept() để nhận client tiếp theo sau khi đã phục vụ xong toàn bộ yêu cầu của client trước đó.

Câu 11: Câu 11: Thành phần nào mở port và lắng nghe kết nối ở phía server?

A. Socket

B. InputStreamReader

C. ServerSocket

D. PrintWriter

➔ Đáp án đúng: C. ServerSocket

Giải thích cốt lõi: Lớp ServerSocket trong Java được sử dụng ở phía Server để ràng buộc (bind) vào một cổng cụ thể và lắng nghe (listen) các yêu cầu kết nối tới.

Câu 12: Câu 12: Lợi ích trực tiếp của fixed thread pool là gì?

A. Giới hạn số thread hoạt động và tái sử dụng thread

B. Tự động mã hóa UTF-8

C. Thay thế hoàn toàn ServerSocket

D. Bảo đảm mọi request đúng cú pháp

➔ Đáp án đúng: A. Giới hạn số thread hoạt động và tái sử dụng thread

Giải thích cốt lõi: Fixed Thread Pool khống chế số lượng luồng tối đa chạy đồng thời để tránh làm tràn RAM/CPU hệ thống, đồng thời tái sử dụng các thread cũ thay vì liên tục tạo mới.

Câu 13: Câu 13: Tham số true trong new PrintWriter(writer, true) có tác dụng gì?

A. Mã hóa UTF-8

B. Chạy server bằng Thread

C. Bật autoFlush cho println(), printf() và format()

D. Đóng socket tự động

➔ Đáp án đúng: C. Bật autoFlush cho println(), printf() và format()

Giải thích cốt lõi: Tham số thứ 2 là boolean autoFlush. Khi truyền true, PrintWriter sẽ tự động đẩy (flush) dữ liệu ra khỏi bộ đệm ngay khi các phương thức println(), printf(), hoặc format() được gọi.

Câu 14: Câu 14: Lỗi IOException của một client nên được xử lý như thế nào?

A. Đóng ServerSocket trước

B. Dừng toàn bộ server

C. Bỏ qua việc đóng Socket

D. Bắt lỗi trong worker và tiếp tục phục vụ client khác

➔ Đáp án đúng: D. Bắt lỗi trong worker và tiếp tục phục vụ client khác

Giải thích cốt lõi: Xử lý lỗi riêng lẻ trong luồng worker tương ứng giúp bảo vệ Server khỏi bị gián đoạn, đảm bảo các kết nối client khác vẫn tiếp tục hoạt động bình thường.

Câu 15: Câu 15: Cách tổ chức request-response nào đúng với ví dụ?

A. Client gửi tất cả request rồi đóng ngay

B. Server gửi request trước

C. Mỗi dòng phải tạo một ServerSocket mới

D. Client gửi một dòng và chờ một dòng phản hồi

➔ Đáp án đúng: D. Client gửi một dòng và chờ một dòng phản hồi

Giải thích cốt lõi: Mô hình tương tác cơ bản dạng Request-Response đồng bộ (Synchronous): Client gửi 1 dòng lệnh/dữ liệu và tạm dừng chờ phản hồi 1 dòng tương ứng từ phía Server.

Câu 16: Câu 1: UDP server vừa nhận request trong biến requestPacket. Để gửi phản hồi đúng client, server nên lấy địa chỉ và port từ đâu?

A. InetAddress.getLocalHost() và port của server

B. Một ServerSocket mới

C. Địa chỉ broadcast và port 0

D. requestPacket.getAddress() và requestPacket.getPort()

➔ Đáp án đúng: D. requestPacket.getAddress() và requestPacket.getPort()

Giải thích cốt lõi: Do UDP không duy trì kết nối cố định, DatagramPacket khi nhận về sẽ chứa kèm thông tin địa chỉ IP và số hiệu cổng (port) của nơi gửi qua hai phương thức getAddress() và getPort().

Câu 17: Câu 2: UDP client gửi yêu cầu trừ tiền và sẽ gửi lại nếu quá thời gian chờ. Thiết kế nào hạn chế việc server thực hiện cùng một giao dịch hai lần?

A. Cho mọi client dùng chung một port nguồn cố định

B. Tăng kích thước buffer nhận

C. Đổi toàn bộ nội dung request thành chữ hoa

D. Gắn mã request duy nhất, lưu kết quả đã xử lý và trả lại kết quả cũ khi request bị lặp

➔ Đáp án đúng: D. Gắn mã request duy nhất, lưu kết quả đã xử lý và trả lại kết quả cũ khi request bị lặp

Giải thích cốt lõi: Đây là cơ chế Idempotency trong thiết kế hệ thống. Bằng cách gán Request ID duy nhất, server có thể kiểm tra xem giao dịch đã được thực hiện chưa để tránh xử lý trùng lặp.

Câu 18: Câu 3: Sau khi gọi socket.setSoTimeout(2000), nếu không có datagram nào đến trong 2 giây thì điều gì xảy ra?

A. JVM tự động khởi động lại socket

B. receive() ném SocketTimeoutException nhưng socket vẫn có thể tiếp tục sử dụng

C. receive() trả về null và socket bị đóng

D. Datagram rỗng được tạo và gửi đến client

➔ Đáp án đúng: B. receive() ném SocketTimeoutException nhưng socket vẫn có thể tiếp tục sử dụng

Giải thích cốt lõi: setSoTimeout đặt thời gian chờ tối đa cho các thao tác đọc. Nếu hết thời gian mà chưa có gói tin đến, Java ném SocketTimeoutException, tuy nhiên đối tượng Socket vẫn mở và sử dụng bình thường.

Câu 19: Câu 4: Điều gì có thể xảy ra khi datagram đến có kích thước lớn hơn vùng buffer của DatagramPacket nhận?

A. Java tự động tạo thêm buffer và giữ nguyên toàn bộ datagram

B. Datagram bị cắt theo kích thước vùng nhận và phần vượt quá không được giữ lại

C. receive() trả về hai DatagramPacket

D. Datagram tự động được chuyển sang TCP

➔ Đáp án đúng: B. Datagram bị cắt theo kích thước vùng nhận và phần vượt quá không được giữ lại

Giải thích cốt lõi: Giao thức UDP xử lý gói tin độc lập. Nếu buffer của DatagramPacket nhỏ hơn kích thước gói tin gửi đến, phần dữ liệu dư thừa sẽ bị xén bỏ (truncated) và mất vĩnh viễn.

Câu 20: Câu 5: Ứng dụng đang gửi một datagram UDP rất lớn qua nhiều mạng trung gian và thường xuyên mất toàn bộ thông điệp. Giải pháp phù hợp nhất khi vẫn phải sử dụng UDP là gì?

A. Dùng println() để router tự ghép các mảnh

B. Chia dữ liệu thành các datagram nhỏ hơn, gắn mã thông điệp và số thứ tự để kiểm tra và ghép lại

C. Tăng datagram đến kích thước tối đa để giảm số lần gửi

D. Bỏ địa chỉ IP đích khỏi DatagramPacket

➔ Đáp án đúng: B. Chia dữ liệu thành các datagram nhỏ hơn, gắn mã thông điệp và số thứ tự để kiểm tra và ghép lại

Giải thích cốt lõi: Các gói tin UDP lớn dễ bị phân mảnh (IP fragmentation) trên đường truyền và bị router loại bỏ nếu vượt quá MTU. Giải pháp là ứng dụng chủ động chia nhỏ gói tin và quản lý thứ tự ghép lại.

Câu 21: Câu 6: Lớp Java nào được dùng để gửi và nhận datagram UDP?

A. NetworkInterface

B. ServerSocket

C. DatagramSocket

D. Socket

➔ Đáp án đúng: C. DatagramSocket

Giải thích cốt lõi: DatagramSocket trong Java đại diện cho điểm cuối (endpoint) để truyền và nhận các gói tin datagram không kết nối theo giao thức UDP.

Câu 22: Câu 7: Một DatagramPacket có buffer 1024 byte vừa nhận gói 100 byte và sẽ được tái sử dụng để nhận gói tiếp theo. Thao tác nào nên thực hiện trước lần receive() kế tiếp?

A. Đổi port của server về 0

B. Gọi packet.getAddress(null)

C. Gọi packet.setLength(buffer.length)

D. Tạo một ServerSocket mới

➔ Đáp án đúng: C. Gọi packet.setLength(buffer.length)

Giải thích cốt lõi: Sau mỗi lần receive(), thuộc tính length của DatagramPacket sẽ tự động bị cập nhật giảm xuống đúng bằng độ dài gói vừa nhận (ở đây là 100 byte). Nếu không reset lại bằng setLength(), lần receive() sau chỉ nhận được tối đa 100 byte.

Câu 23: Câu 8: Cách nào giải mã chính xác chuỗi UTF-8 từ một DatagramPacket vừa nhận?

A. new String(packet.getData(), packet.getOffset(), packet.getLength(), StandardCharsets.UTF_8)

B. packet.toString()

C. new String(packet.getData())

D. new String(packet.getData(), 0, packet.getData().length)

➔ Đáp án đúng: A. new String(packet.getData(), packet.getOffset(), packet.getLength(), StandardCharsets.UTF_8)

Giải thích cốt lõi: Chuỗi mã hóa UTF-8 cần được đọc chính xác từ vị trí bắt đầu (getOffset()) với độ dài thực tế của dữ liệu nhận được (getLength()) thay vì dùng toàn bộ mảng đệm (getData().length).

Câu 24: Câu 9: Theo mặc định, điều gì xảy ra khi gọi socket.receive(packet) nhưng chưa có datagram đến?

A. Socket tự động đóng

B. Phương thức tiếp tục chờ cho đến khi nhận được datagram

C. Một DatagramSocket mới được tạo

D. Phương thức trả về null ngay lập tức

➔ Đáp án đúng: B. Phương thức tiếp tục chờ cho đến khi nhận được datagram

Giải thích cốt lõi: Mặc định, phương thức receive() của DatagramSocket là thao tác khóa (blocking operation). Nó sẽ tạm dừng luồng thi hành cho đến khi có một gói tin được gửi tới.

Câu 25: Câu 10: Phát biểu nào mô tả đúng nhất về UDP?

A. UDP chỉ được sử dụng trong mạng cục bộ

B. UDP là giao thức không kết nối và truyền dữ liệu theo datagram

C. UDP phải thiết lập kết nối trước khi truyền dữ liệu

D. UDP bảo đảm mọi gói tin đến đúng thứ tự

➔ Đáp án đúng: B. UDP là giao thức không kết nối và truyền dữ liệu theo datagram

Giải thích cốt lõi: UDP (User Datagram Protocol) là giao thức truyền dữ liệu độc lập dạng Datagram, không cần thiết lập hay duy trì kết nối trước khi gửi.

Câu 26: Câu 11: Đoạn lệnh nào tạo một UDP server lắng nghe tại port 9876?

A. new DatagramSocket()

B. new ServerSocket(9876)

C. new DatagramSocket(9876)

D. new Socket(9876)

➔ Đáp án đúng: C. new DatagramSocket(9876)

Giải thích cốt lõi: Để lắng nghe các gói tin UDP gửi đến tại một cổng cố định trên server, ta khởi tạo DatagramSocket bằng cách truyền số hiệu port vào constructor: new DatagramSocket(9876).

Câu 27: Câu 12: Một buffer 1024 byte được dùng để nhận UDP, nhưng datagram thực tế chỉ có 120 byte. Giá trị nào cho biết số byte thực nhận?

A. socket.getReceiveBufferSize()

B. packet.getLength()

C. buffer.length

D. packet.getPort()

➔ Đáp án đúng: B. packet.getLength()

Giải thích cốt lõi: Phương thức getLength() trên đối tượng DatagramPacket trả về chiều dài dữ liệu thực tế (tính bằng byte) được ghi vào đệm trong lần nhận đó.

Câu 28: Câu 1: UDP không cung cấp bảo đảm nào sau đây?

A. Cho phép ứng dụng gửi dữ liệu không cần thiết lập kết nối

B. Bảo toàn ranh giới của từng datagram

C. Dữ liệu được nhận đầy đủ và đúng thứ tự

D. Gửi dữ liệu đến một port đích

➔ Đáp án đúng: C. Dữ liệu được nhận đầy đủ và đúng thứ tự

Giải thích cốt lõi: UDP là giao thức không tin cậy (unreliable), không đảm bảo dữ liệu đến nơi, không chống mất gói và không sắp xếp thứ tự các gói tin đến.

Câu 29: Câu 2: Đối tượng nào chứa dữ liệu, độ dài và thông tin địa chỉ của một gói UDP trong Java?

A. BufferedReader

B. ServerSocket

C. DatagramPacket

D. PrintWriter

➔ Đáp án đúng: C. DatagramPacket

Giải thích cốt lõi: Lớp DatagramPacket đóng vai trò là container chứa mảng byte dữ liệu, độ dài, địa chỉ IP và cổng (port) của nguồn hoặc đích trong giao tiếp UDP.

Câu 30: Câu 3: Giả sử data là mảng byte, address là InetAddress và port là port đích. Cách tạo gói gửi nào đúng?

A. new DatagramPacket(data, address)

B. new DatagramSocket(data, port)

C. new Socket(address, data.length)

D. new DatagramPacket(data, data.length, address, port)

➔ Đáp án đúng: D. new DatagramPacket(data, data.length, address, port)

Giải thích cốt lõi: Constructor đầy đủ để tạo DatagramPacket nhằm gửi đi bao gồm: mảng byte dữ liệu, chiều dài dữ liệu, địa chỉ IP đích (InetAddress) và số hiệu port đích.

Câu 31: Câu 4: Client thực hiện hai lệnh:

A. Lệnh flush() tự động thêm ký tự xuống dòng giữa hai chuỗi.

B. Server chắc chắn nhận được hai thông điệp riêng biệt.

C. Server nhận một dòng byte; ranh giới giữa HELLO và WORLD không được TCP bảo đảm.

D. TCP chỉ gửi chuỗi WORLD vì chuỗi HELLO bị ghi đè.

➔ Đáp án đúng: C. Server nhận một dòng byte; ranh giới giữa HELLO và WORLD không được TCP bảo đảm.

Giải thích cốt lõi: TCP hoạt động dựa trên luồng byte liên tục (byte stream) và không giữ ranh giới thông điệp (no message boundaries). Hai chuỗi in ra nối liền thành 'HELLOWORLD' nếu không có ký tự phân tách.

Câu 32: Câu 5: Trong chương trình TCP, phương thức sau trả về thành phần nào?

A. Địa chỉ IP của máy đang kết nối.

B. Luồng gửi dữ liệu đến đầu bên kia.

C. Danh sách các client đang kết nối.

D. Luồng đọc dữ liệu do đầu bên kia gửi đến.

➔ Đáp án đúng: D. Luồng đọc dữ liệu do đầu bên kia gửi đến.

Giải thích cốt lõi: getInputStream() trên đối tượng Socket trả về luồng vào (InputStream) dùng để nhận/đọc dữ liệu được truyền tới từ phía socket đối diện.

Câu 33: Câu 6: Xét đoạn mã:

A. Luồng đầu vào đạt EOF do đầu bên kia đóng luồng hoặc kết nối.

B. Dữ liệu chưa đến trong một mili giây.

C. Client gửi một dòng rỗng.

D. Server nhận được chuỗi QUIT.

➔ Đáp án đúng: A. Luồng đầu vào đạt EOF do đầu bên kia đóng luồng hoặc kết nối.

Giải thích cốt lõi: Phương thức readLine() chỉ trả về null khi gặp điểm cuối luồng (EOF - End Of Stream), xảy ra khi đầu bên kia đóng luồng hoặc ngắt kết nối socket.

Câu 34: Câu 7: Xét đoạn mã:

A. Thread pool bảo đảm TCP giữ nguyên ranh giới thông điệp.

B. Mọi client vẫn được xử lý tuần tự trên luồng chính.

C. Mỗi worker tạo một ServerSocket trên cùng port.

D. Luồng chính nhận kết nối và giao từng Socket cho worker xử lý.

➔ Đáp án đúng: D. Luồng chính nhận kết nối và giao từng Socket cho worker xử lý.

Giải thích cốt lõi: Đoạn mã mô hình Reactor/ThreadPool server: luồng chính liên tục accept() kết nối và đẩy tác vụ xử lý client socket vào ThreadPool.

Câu 35: Câu 8: Xét đoạn mã:

A. Tự động đóng Socket sau khi gửi.

B. Tạo một thread mới để gửi dữ liệu.

C. Tự động mã hóa dữ liệu thành UTF-8.

D. Tự động đẩy dữ liệu khi gọi println().

➔ Đáp án đúng: D. Tự động đẩy dữ liệu khi gọi println().

Giải thích cốt lõi: Tham số thứ hai (autoFlush = true) trong PrintWriter đảm bảo dữ liệu được đẩy (flush) ngay lập tức khi gọi các hàm println(), printf() hoặc format().

Câu 36: Câu 9: Lớp Java nào được sử dụng làm điểm cuối để gửi và nhận các gói UDP?

A. Socket

B. ServerSocket

C. DatagramSocket

D. BufferedReader

➔ Đáp án đúng: C. DatagramSocket

Giải thích cốt lõi: DatagramSocket là lớp cốt lõi trong Java dùng để thiết lập cổng gửi và nhận các gói tin Datagram theo giao thức UDP.

Câu 37: Câu 10: Xét dòng lệnh:

A. Tạo một ServerSocket mới.

B. Đóng port đang lắng nghe của server.

C. Chờ client kết nối và trả về Socket của phiên kết nối.

D. Nhận ngay một dòng dữ liệu từ client.

➔ Đáp án đúng: C. Chờ client kết nối và trả về Socket của phiên kết nối.

Giải thích cốt lõi: ServerSocket.accept() là phương thức khóa (blocking), chờ đợi đến khi có client kết nối tới và trả về đối tượng Socket đại diện cho kênh truyền riêng với client đó.

Câu 38: Câu 11: Xét đoạn mã:

A. receive() ném SocketTimeoutException và socket vẫn có thể sử dụng.

B. Socket tự động chuyển sang TCP.

C. receive() trả về null.

D. DatagramSocket tự động bị đóng.

➔ Đáp án đúng: A. receive() ném SocketTimeoutException và socket vẫn có thể sử dụng.

Giải thích cốt lõi: setSoTimeout thiết lập thời gian chờ tối đa cho lệnh receive(). Nếu quá thời gian, ngoại lệ SocketTimeoutException được ném ra nhưng socket vẫn giữ nguyên trạng thái mở.

Câu 39: Câu 12: Vì sao nên giải mã datagram bằng đoạn mã sau?

A. Để chỉ giải mã vùng byte thực nhận bằng đúng bảng mã UTF-8.

B. Để tăng kích thước buffer sau mỗi lần nhận.

C. Để tự động gửi phản hồi cho client.

D. Để chuyển giao thức UDP thành TCP.

➔ Đáp án đúng: A. Để chỉ giải mã vùng byte thực nhận bằng đúng bảng mã UTF-8.

Giải thích cốt lõi: Đoạn mã này đảm bảo chỉ chuyển đổi chính xác phần dữ liệu thực tế nằm trong gói tin nhận được (dựa vào offset và length) thành chuỗi ký tự theo chuẩn UTF-8.

Câu 40: Câu 13: UDP client gửi yêu cầu trừ tiền và gửi lại request nếu hết thời gian chờ. Giải pháp nào giúp server không thực hiện cùng một giao dịch hai lần?

A. Gắn mã request duy nhất và lưu kết quả của request đã xử lý.

B. Gửi mỗi request thành một datagram lớn hơn.

C. Tăng kích thước buffer nhận.

D. Cho tất cả client dùng chung một port nguồn.

➔ Đáp án đúng: A. Gắn mã request duy nhất và lưu kết quả của request đã xử lý.

Giải thích cốt lõi: Cơ chế chống lặp giao dịch (Idempotency) bằng cách kiểm tra Request ID đã được xử lý lưu trong cache/database hay chưa.

Câu 41: Câu 14: Xét đoạn mã phía UDP server:

A. Xác định địa chỉ và port của server.

B. Xác định kích thước tối đa của datagram.

C. Xác định địa chỉ IP và port của client đã gửi request.

D. Tạo kết nối TCP mới đến client.

➔ Đáp án đúng: C. Xác định địa chỉ IP và port của client đã gửi request.

Giải thích cốt lõi: request.getAddress() và request.getPort() trích xuất thông tin địa chỉ nguồn từ gói tin request của client để gói phản hồi (response) gửi về đúng địa chỉ.

Câu 42: Câu 15: Lớp Java nào được sử dụng để mở port và lắng nghe kết nối TCP ở phía server?

A. Socket

B. DatagramSocket

C. DatagramPacket

D. ServerSocket

➔ Đáp án đúng: D. ServerSocket

Giải thích cốt lõi: ServerSocket dùng ở phía Server để bind vào một port cụ thể và lắng nghe kết nối từ các TCP client.

Câu 43: Câu 16: Ứng dụng bắt buộc sử dụng UDP nhưng cần phát hiện mất gói, gói trùng và hỗ trợ gửi lại. Giải pháp nào phù hợp nhất?

A. Bổ sung số thứ tự hoặc mã request, ACK, timeout và cơ chế retransmission.

B. Cho tất cả client sử dụng chung một port nguồn cố định.

C. Tăng mọi datagram đến kích thước tối đa của UDP.

D. Gọi accept() trên DatagramSocket trước mỗi lần nhận.

➔ Đáp án đúng: A. Bổ sung số thứ tự hoặc mã request, ACK, timeout và cơ chế retransmission.

Giải thích cốt lõi: Vì UDP không cung cấp độ tin cậy, ứng dụng phải tự xây dựng cơ chế báo nhận (ACK), timeout và truyền lại (retransmission) tương tự như TCP ở tầng ứng dụng.

Câu 44: Câu 17: Một DatagramPacket có buffer 1024 byte vừa nhận datagram dài 100 byte. Trước khi tái sử dụng packet để nhận datagram tiếp theo, nên thực hiện lệnh nào?

A. packet.setPort(0);

B. packet.getData().length = 100;

C. socket.connect(null);

D. packet.setLength(buffer.length);

➔ Đáp án đúng: D. packet.setLength(buffer.length);

Giải thích cốt lõi: Sau khi nhận gói tin nhỏ hơn buffer, chiều dài packet bị thu hẹp còn 100 byte. Cần gọi setLength(buffer.length) để khôi phục kích thước bộ đệm cho lần nhận kế tiếp.

Câu 45: Câu 18: Phát biểu nào phân biệt đúng TCP và UDP?

A. UDP luôn phù hợp hơn TCP vì có tốc độ cao hơn.

B. TCP bảo toàn ranh giới thông điệp; UDP cung cấp dòng byte liên tục.

C. TCP cung cấp dòng byte tin cậy; UDP truyền datagram không kết nối.

D. Cả TCP và UDP đều phải thiết lập kết nối trước khi truyền.

➔ Đáp án đúng: C. TCP cung cấp dòng byte tin cậy; UDP truyền datagram không kết nối.

Giải thích cốt lõi: TCP là giao thức hướng kết nối, tin cậy, truyền dữ liệu theo dòng byte (byte stream); trong khi UDP là giao thức không kết nối, truyền gói tin độc lập (datagram) không bảo đảm tin cậy.

⚖️ PHẦN IV: MA TRẬN TRADEOFF SO SÁNH CỐT LÕI (BẢN ĐỒ KHÁI NIỆM)

1. Ma trận 1: TCP vs UDP (Bản chất truyền dẫn)

Tiêu chí	TCP (`Socket`, `ServerSocket`)	UDP (`DatagramSocket`, `DatagramPacket`)
Kiểu kết nối	**Hướng kết nối** (*Connection-oriented*): Bắt tay 3 bước (SYN, SYN-ACK, ACK).	**Phi kết nối** (*Connectionless*): Bắn gói tin đi ngay, không bắt tay, không duy trì phiên.
Độ tin cậy	**Tuyệt đối**: Đảm bảo tới nơi, đúng thứ tự, có cờ ACK, tự động retransmission nếu mất gói.	**Không tin cậy** (*Best-effort*): Có thể mất gói, trùng lặp hoặc đến lộn xộn thứ tự.
Biên thông điệp	**Không bảo toàn** (*Byte stream*): Dữ liệu truyền liên tục, phải tự tạo ranh giới bằng **\n** hoặc gửi length.	**Bảo toàn hoàn hảo** (*Message boundary*): Mỗi datagram là một gói tin nguyên vẹn, độc lập.
Cơ chế Timeout	Socket đóng khi đứt mạng hoặc nhận gói TCP RST/FIN.	Mặc định **receive() chờ vô hạn**. Bắt buộc cấu hình **socket.setSoTimeout(ms)**.
Quản lý Socket	Cần 2 lớp: **ServerSocket** (lắng nghe) và **Socket** (giao tiếp).	Dùng duy nhất 1 lớp **DatagramSocket** cho cả gửi và nhận ở cả 2 phía.
Ứng dụng	Web (HTTP/HTTPS), Tải file, Email, Giao dịch ngân hàng.	DNS, Video Call, Voice chat, Livestream, Game thời gian thực.

2. Ma trận 2: Kiến trúc Server (Tuần tự vs ThreadPool)

Tiêu chí	Server Tuần Tự (Sequential)	Server Đa Luồng (ThreadPool Master-Worker)
Cách xử lý	Đơn luồng: Phục vụ xong 1 client mới quay lại gọi **accept()**.	Đa luồng: Luồng chính chỉ **accept()**, giao Socket cho Worker trong pool.
Số client đồng thời	Đúng 1 client tại một thời điểm.	N client đồng thời (N là kích thước **FixedThreadPool**).
Rủi ro lớn nhất	**Head-of-Line Blocking**: 1 client lag/treo làm tê liệt toàn bộ server.	Quá tải hàng đợi nếu số lượng request vượt xa năng lực xử lý.
Rủi ro về RAM	Rất thấp, bộ nhớ cố định.	An toàn tuyệt đối với **FixedThreadPool(50)**. Tránh bẫy tràn RAM **OutOfMemory**.
Độ phức tạp code	Đơn giản, 1 vòng lặp **while(true)**.	Cần **ExecutorService**, xử lý đồng bộ và **ConcurrentHashMap**.

3. Ma trận 3: Hệ thống Luồng I/O trong Lập trình Mạng Java

Nhóm luồng	Các lớp cốt lõi	Kiểu dữ liệu xử lý	Cơ chế & Bẫy cần nhớ
Byte Stream	**InputStream**, **OutputStream**	Byte thô (**byte[]**, **int 0-255**)	Gốc của mọi socket (**socket.getInputStream()**).
Character Stream	**InputStreamReader**, **OutputStreamWriter**	Ký tự Unicode (char)	Cầu nối chuyển byte ➔ char. **Bắt buộc chỉ định `StandardCharsets.UTF_8`**.
Buffered Stream	**BufferedReader**, **BufferedWriter**	Bộ đệm văn bản	Cung cấp **readLine()**. Đọc tới khi gặp **\n** hoặc EOF (**null**).
Print Stream	**PrintWriter**	Văn bản có định dạng	**new PrintWriter(writer, true)** ➔ Bật **`autoFlush`** cho **println()**, **printf()**.
Data Stream	**DataInputStream**, **DataOutputStream**	Kiểu nguyên thủy (**int**, **double**, **boolean**, **UTF**)	**readInt()**, **writeDouble()**, **writeUTF()**. **BẮT BUỘC gọi `.flush()` sau khi ghi!**

4. Ma trận 4: RMI & JDBC (Tích hợp Hệ thống)

RMI (Remote Method Invocation):

Interface từ xa: Bắt buộc **extends java.rmi.Remote**. Mọi phương thức **bắt buộc ném `java.rmi.RemoteException`**.

Lớp thực thi (Impl): extends java.rmi.server.UnicastRemoteObject implements MyInterface. Constructor phải ném **RemoteException**.

Đăng ký Server: LocateRegistry.createRegistry(1099); Naming.rebind("rmi://localhost/Service", obj);

Client tra cứu: MyInterface stub = (MyInterface) Naming.lookup("rmi://localhost/Service");

JDBC (5 Bước chuẩn):

1. Nạp Driver (tự động từ JDBC 4.0).
2. Mở kết nối: **Connection conn = DriverManager.getConnection(url, user, pass);**
3. Tạo câu lệnh: **PreparedStatement ps = conn.prepareStatement("SELECT * FROM tbl WHERE id = ?");**
4. Thực thi: **ps.setInt(1, id); ResultSet rs = ps.executeQuery();** (hoặc **ps.executeUpdate()** cho INSERT/UPDATE/DELETE).
5. Đóng tài nguyên: Đóng theo thứ tự ngược lại **rs ➔ ps ➔ conn** (hoặc dùng **try-with-resources**).

🛠️ PHẦN V: NGÂN HÀNG 10 SỰ CỐ MẠNG (CHUẨN 3 BƯỚC: NGUYÊN NHÂN ➔ HẬU QUẢ ➔ KHẮC PHỤC)

Sự cố 1: Client gửi dữ liệu nhưng Server không nhận được gì

Nguyên nhân cốt lõi: Quên gọi **.flush()** trên **DataOutputStream** / **BufferedWriter**, hoặc bên gửi dùng **print()** thay vì **println()** khiến dữ liệu đọng lại trong bộ đệm RAM chưa đẩy lên luồng mạng.

Hậu quả trực tiếp: Server bị block vô hạn tại **in.readLine()** hoặc **in.readInt()**, gây treo toàn bộ phiên kết nối.

Giải pháp khắc phục: Luôn gọi **out.flush()** ngay sau khi ghi dữ liệu; đối với **PrintWriter**, luôn kích hoạt constructor tự động xả **new PrintWriter(writer, true)** và dùng **println()**.

Sự cố 2: Một client mạng chậm làm toàn bộ hệ thống bị treo cứng

Nguyên nhân cốt lõi: Server thiết kế theo kiến trúc tuần tự (Single-threaded). Luồng chính bị khóa (blocking) trong hàm phục vụ client hiện tại và không thể quay lại vòng lặp gọi **accept()**.

Hậu quả trực tiếp: Lỗi nghẽn đầu hàng (Head-of-Line Blocking); tất cả các client kết nối sau bị treo ở hàng đợi kết nối của hệ điều hành.

Giải pháp khắc phục: Chuyển sang mô hình Master-Worker: Luồng chính chỉ làm duy nhất việc **Socket client = server.accept();** rồi bàn giao ngay cho Worker trong ThreadPool xử lý song song (**pool.submit(() -> handle(client));**).

Sự cố 3: Server bị sập vì lỗi OutOfMemoryError khi lượng truy cập tăng

Nguyên nhân cốt lõi: Tạo luồng thủ công không giới hạn bằng **new Thread(() -> handle(client)).start()** cho mỗi kết nối đến.

Hậu quả trực tiếp: Mỗi Thread trong Java chiếm dụng từ 512KB đến 1MB vùng nhớ Stack. Khi có hàng nghìn kết nối đồng thời, JVM cạn kiệt RAM và crash hoàn toàn.

Giải pháp khắc phục: Sử dụng **Executors.newFixedThreadPool(50)**. Giới hạn số luồng tối đa hoạt động đồng thời và tái sử dụng luồng (thread reuse).

Sự cố 4: Gói UDP sau bị mất đuôi dữ liệu khi tái sử dụng DatagramPacket

Nguyên nhân cốt lõi: Sau khi nhận gói tin có kích thước nhỏ (ví dụ 100 byte), thuộc tính nội bộ **length** của **DatagramPacket** tự động bị gán thành 100.

Hậu quả trực tiếp: Khi dùng lại gói đó để nhận gói tin kế tiếp có kích thước lớn hơn (ví dụ 500 byte), hệ điều hành sẽ cắt cụt dữ liệu (Packet Truncation), làm mất 400 byte phía sau.

Giải pháp khắc phục: Luôn gọi **packet.setLength(buffer.length);** trước mỗi lần gọi **socket.receive(packet);** tiếp theo.

Sự cố 5: Client giải mã chuỗi UTF-8 UDP bị dính ký tự rác

Nguyên nhân cốt lõi: Giải mã bằng lệnh **new String(packet.getData())** hoặc dùng **buffer.length** thay vì độ dài thực tế.

Hậu quả trực tiếp: Chuỗi kết quả bị độn thêm hàng trăm byte rác còn sót lại từ bộ đệm ban đầu.

Giải pháp khắc phục: Bắt buộc giải mã đúng 4 tham số: **new String(packet.getData(), packet.getOffset(), packet.getLength(), StandardCharsets.UTF_8);**

Sự cố 6: Giao dịch trừ tiền qua UDP bị thực hiện 2 lần

Nguyên nhân cốt lõi: Mạng lag làm mất gói tin phản hồi ACK. Client hết thời gian timeout nên tự động truyền lại (retransmit) request trừ tiền. UDP không có cơ chế phát hiện gói trùng.

Hậu quả trực tiếp: Server nhận được request thứ hai và trừ tiền tài khoản khách hàng lần nữa.

Giải pháp khắc phục: Thiết kế tính lũy thừa (Idempotency): Gắn Request-ID duy nhất (UUID) cho mỗi giao dịch; Server lưu kết quả đã xử lý vào bộ nhớ đệm (Cache). Khi gặp lại Request-ID cũ, chỉ trả về kết quả cũ mà không thực hiện lại giao dịch.

Sự cố 7: Đóng Stream làm đóng luôn cả Socket

Nguyên nhân cốt lõi: Trong Java, gọi **in.close()** hoặc **out.close()** trên các stream bọc ngoài sẽ tự động kích hoạt đóng **Socket** bên dưới.

Hậu quả trực tiếp: Phiên làm việc bị ngắt đột ngột, các thao tác đọc/ghi tiếp theo sẽ ném **SocketException: Socket is closed**.

Giải pháp khắc phục: Không đóng các stream bọc ngoài khi phiên giao tiếp chưa kết thúc; chỉ đóng **Socket** ở khối **finally** hoặc dùng **try-with-resources** bọc toàn bộ phiên làm việc.

Sự cố 8: SocketTimeoutException bị hiểu lầm là Socket bị chết

Nguyên nhân cốt lõi: Khi gọi **socket.setSoTimeout(2000)** và quá 2 giây không có gói tin, phương thức **receive()** ném ngoại lệ **SocketTimeoutException**.

Hậu quả trực tiếp: Lập trình viên bắt ngoại lệ rồi vội vã đóng socket, làm gián đoạn chương trình.

Giải pháp khắc phục: Ngoại lệ **SocketTimeoutException** chỉ báo hiệu hết thời gian chờ, socket hoàn toàn không bị đóng. Cần đặt lời gọi trong khối **try-catch**, bắt ngoại lệ để xử lý logic thử lại và tiếp tục vòng lặp nhận tin bình thường.

Sự cố 9: Lỗi BindException: Address already in use

Nguyên nhân cốt lõi: Cổng port dự định mở (ví dụ 5000) đang bị chiếm dụng bởi một tiến trình khác, hoặc tiến trình cũ vừa tắt nhưng hệ điều hành đang giữ port ở trạng thái **TIME_WAIT**.

Hậu quả trực tiếp: new ServerSocket(port) thất bại ngay lập tức, Server không thể khởi động.

Giải pháp khắc phục: Đổi sang số cổng khác, hoặc tìm và kill tiến trình đang chiếm port (**lsof -i :port → kill -9 PID**), hoặc bật cờ tái sử dụng địa chỉ: **serverSocket.setReuseAddress(true);**.

Sự cố 10: Dữ liệu TCP bị dính chùm vào nhau (Packet Glued / Sticky)

Nguyên nhân cốt lõi: TCP là Byte-stream protocol truyền liên tục, không có khái niệm thông điệp rời rạc. Nếu bên gửi gọi **out.print("HELLO"); out.print("WORLD"); out.flush();**, TCP có thể gom chung vào một segment.

Hậu quả trực tiếp: Bên nhận đọc một lần được cả chuỗi **HELLOWORLD**, mất ranh giới giữa 2 lệnh.

Giải pháp khắc phục: Tầng ứng dụng phải tự định nghĩa giao thức phân tách: dùng ký tự xuống dòng **\n** kết hợp **println()** / **readLine()**, hoặc gửi 4 byte độ dài thông điệp ở phần đầu (Length-prefixed Framing).